LAPORAN TUGAS BESAR 1 PEMBELAJARAN MESIN

IF3270 Pembelajaran Mesin

Implementasi Feedforward Neural Network

Disusun Oleh:

Raka Daffa Iftikhaar (13523018)
Muhammad Fithra Rizki (13523049)
Muhammad Timur Kanigara (13523055)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG

2026

DAFTAR ISI

DAFTAR ISI 1
BAB I 2
DESKRIPSI PERSOALAN 2
BAB II 3
PEMBAHASAN 3
2.1. Penjelasan Implementasi 3
2.1.1 Deskripsi Kelas dan Atribut 3
2.1.2. Forward Propagation 4
2.1.3. Backward Propagation dan Weight Update 5
2.1.4. Implementasi Algoritma Optimasi dan Fitur Lanjutan (Bonus) 6
2.2. Hasil Pengujian 7
2.2.1 Pengaruh Depth dan Width 7
2.2.2 Pengaruh Fungsi Aktivasi 9
2.2.3 Pengaruh Learning Rate 14
2.2.4 Pengaruh Inisialisasi Bobot 19
2.2.5 Pengaruh Regularisasi 21
2.2.6 Perbandingan Dengan Library sklearn 24
2.2.7 Perbandingan Adam Optimizer vs Gradient Descent [BONUS] 25
2.2.8 Perbandingan Adam Optimizer vs Gradient Descent [BONUS] 27
BAB III 30
KESIMPULAN DAN SARAN 30
3.1. Kesimpulan 30
3.2. Saran 30
Pembagian Tugas Tiap Anggota Kelompok 32
Referensi 33

BAB I
DESKRIPSI PERSOALAN

Pada tugas besar ini, persoalan utama yang diselesaikan adalah pembangunan dan pengujian model *Feedforward Neural Network* (FFNN) dari awal (*from scratch*) tanpa menggunakan pustaka *machine learning* tingkat tinggi untuk inti algoritmanya. Model yang dibangun kemudian diimplementasikan untuk menyelesaikan masalah klasifikasi pada dataset *Global Student Placement & Salary*.

Dataset ini memuat informasi demografis dan rekam jejak akademis mahasiswa, yang mencakup berbagai fitur prediktor (input) seperti [Sebutkan 3-4 fitur utama, misalnya: nilai IPK, program studi, jenis kelamin, dan pengalaman kerja]. Tujuan utama dari pemodelan ini adalah untuk memprediksi target kelas (output), yaitu [Sebutkan targetnya, misalnya: status penempatan kerja (*Placement Status*) atau rentang gaji (*Salary*)] dari seorang mahasiswa berdasarkan fitur-fitur yang telah disebutkan. Persoalan ini merupakan bentuk penyelesaian masalah klasifikasi [sebutkan klasifikasi biner atau multikelas tergantung targetnya] menggunakan arsitektur jaringan saraf tiruan.

Karena dataset mentah memiliki format tipe data yang bervariasi dan rentang nilai yang berbeda-beda, data tidak dapat langsung diumpankan ke dalam model jaringan saraf. Oleh karena itu, penyelesaian persoalan ini juga mencakup tahapan pra-pemrosesan data (*data preprocessing*). Tahapan ini meliputi transformasi data kategorikal menjadi bentuk numerik representatif menggunakan teknik *label encoding* atau *one-hot encoding*, serta proses standardisasi nilai fitur numerik agar memiliki skala yang seragam. Standardisasi ini krusial untuk mencegah dominasi fitur dengan rentang nilai besar terhadap perhitungan gradien selama proses pelatihan model, sekaligus mempercepat tercapainya konvergensi saat *gradient descent* beroperasi.

Melalui penyelesaian masalah ini, diharapkan model FFNN yang diimplementasikan secara fundamental mampu mengenali pola non-linear pada data mahasiswa tersebut, serta diuji ketahanannya terhadap gejala *overfitting* maupun *vanishing gradient* melalui berbagai skenario modifikasi *hyperparameter*.

BAB II
PEMBAHASAN

2.1. Penjelasan Implementasi

2.1.1 Deskripsi Kelas dan Atribut

1. **Kelas Tensor dan Value (autograd.py) [BONUS]**

   Kelas ini merupakan fondasi utama sistem autograd (Diferensiasi Otomatis). `Value` memproses komputasi pada tingkat skalar, sedangkan `Tensor` merupakan abstraksi array multidimensi (NumPy) yang memiliki kemampuan melacak riwayat operasinya.

   Atribut Utama:

   - `data`: Menyimpan matriks nilai numerik (representasi parameter atau hasil aktivasi).
   - `grad`: Menyimpan nilai matriks gradien (turunan) terhadap komputasi akhir.
   - `_children` dan `_prev`: Menyimpan referensi/pointer ke objek Tensor induknya yang membentuk graf komputasi berarah.
   - `_backward`: Menyimpan fungsi closure yang mendefinisikan aturan turunan (chain rule) spesifik untuk operasi matematika terkait (seperti perkalian matriks, eksponensial, atau penjumlahan).

   Method Utama:

   - `backward`() untuk memicu pencarian topologis (topological sort) dari ujung graf (nilai loss) hingga ke daun graf (bobot awal) untuk menyebarkan nilai gradien mundur.

2. **Kelas Loss (losses.py)**

   Kelas dasar yang mengevaluasi selisih kerugian (error) antara prediksi model dan label aktual.

   - Turunan `Loss`: Diimplementasikan dalam bentuk MSE (Mean Squared Error) untuk regresi, serta BCE (Binary Cross-Entropy) dan CCE (Categorical Cross-Entropy) untuk klasifikasi.
   - Method Utama: Method `forward`() menerima parameter tebakan berwujud Tensor dan mengembalikan skalar nilai kerugian yang juga dibungkus sebagai Tensor ujung (root node). Khusus pada fungsi logaritmik kompleks seperti BCE dan CCE, kelas kerugian ini menyisipkan fungsi hook `_backward`() modifikasi untuk menstabilkan kalkulasi gradien pembagian dan mencegah Not-a-Number (NaN).

3. **Kelas Activation (activation.py)**
   Kelas dasar (base class) yang menerapkan fungsi aktivasi non-linear pada lapisan jaringan. Dengan adanya arsitektur autograd, kelas-kelas aktivasi ini dirancang ulang agar tidak lagi memerlukan perhitungan kalkulus turunan secara manual (tidak ada method `backward`()).
   - Turunan `Activation`: Meliputi fungsi standar Linear, ReLU, Sigmoid, Tanh, dan Softmax. Method `forward`() pada kelas-kelas ini memanipulasi objek Tensor (contoh: pemanfaatan boolean mask pada ReLU atau manipulasi .exp() pada Sigmoid) sehingga operasi turunannya terekam secara otomatis ke dalam graf komputasi.
   - Aktivasi Lanjutan [FITUR BONUS]: Diimplementasikan pula turunan kelas `LeakyReLU` yang berfungsi mencegah fenomena Dying ReLU dengan mengizinkan gradien kecil (parameter α) pada input negatif, serta kelas ELU (Exponential Linear Unit) yang memberikan peluruhan eksponensial logaritmik agar rata-rata aktivasi lebih mendekati nol.
4. **Kelas DenseLayer (layer.py)**
   Merepresentasikan satu lapisan komputasi Fully Connected.
   - Atribut Utama: Parameter bobot W dan bias b yang diinisialisasi secara langsung sebagai objek Tensor dengan penanda `requires_grad=True`. Atribut ini memuat pelacakan momentum untuk optimasi lanjutan, yakni `m_W`, `v_W`, `m_b`, `v_b`, dan counter iterasi `t_adam`.
   - Method Utama: `forward`() untuk mengeksekusi propagasi maju ($Z = XW + b$), `update`() untuk memperbarui bobot berdasarkan nilai penalti regularisasi dan algoritma optimasi, serta `zero_grad`() untuk mereset riwayat gradien antar-batch.
5. **Kelas FFNN (ffnn.py)**
   Kelas orkestrator yang membungkus dan mengelola siklus hidup komputasi seluruh lapisan jaringan. Kelas ini menyimpan daftar instansiasi objek `DenseLayer` dalam sebuah list, fungsi objektif (`loss_fn`), serta memfasilitasi parameter penggerak algoritma optimasi (meliputi lr, hyperparameter Adam beta1 dan beta2, serta bobot penalti regularisasi L1/L2 lambda_).

2.1.2. Forward Propagation

Proses propagasi maju (forward propagation) pada arsitektur ini tidak sekadar melakukan komputasi linier aljabar biasa, melainkan bertindak sebagai mesin

pembangun graf komputasi dinamis (Dynamic Computational Graph) berbentuk Directed Acyclic Graph (DAG). Ketika matriks input X (berdimensi batch size x jumlah fitur) dialirkan ke dalam DenseLayer, operasi transformasi affine $Z = XW + b$ dieksekusi secara berurutan. Evaluasi matematis ini difasilitasi oleh mekanisme magic methods bawaan Python yang telah di-override pada objek Tensor (seperti __matmul__ untuk perkalian titik matriks dan __add__ untuk penambahan bias secara broadcasting).

Setiap kali sebuah operasi matematika terjadi, mesin tidak hanya menghitung nilai numeriknya, tetapi juga menginstansiasi sebuah objek Tensor baru (misalnya Z) sebagai node hasil. Objek baru ini secara otomatis menyimpan tautan referensi ke objek-objek pembentuknya (yaitu node X, W, dan b) ke dalam memori melalui atribut _children. Lebih jauh lagi, node tersebut membungkus fungsi analitik lokal (direpresentasikan sebagai atribut closure _backward()) yang berisi instruksi spesifik mengenai cara menghitung turunan parsial (local gradient) dari operasi spesifik tersebut.

Setelah transformasi linier selesai, nilai matriks Z kemudian dilewatkan pada lapisan pemetaan non-linier melalui kelas fungsi aktivasi (seperti ReLU, Sigmoid, atau Tanh). Kelas aktivasi ini juga akan memproduksi objek Tensor baru (A) beserta rekam jejak operasinya, membentuk rantai graf yang terus memanjang hingga akhirnya bermuara menjadi matriks probabilitas tebakan (prediksi akhir) di lapisan output. Pendekatan dinamis ini membebaskan arsitektur dari batasan topologi yang kaku, memungkinkan eksperimen dengan berbagai kedalaman dan lebar layer secara fleksibel tanpa perlu mengubah rumusan matematis secara manual.

2.1.3. Backward Propagation dan Weight Update

Backward Propagation dijalankan setelah perhitungan loss function dari output layer saat selesai dikerjakan untuk menghitung gradien komputasi menggunakan chain rule.

Pada awalnya, kelas Loss mengembalikan matriks gradien pertama (dA). Matriks dA ini kemudian dijalankan pada urutan terbalik melalui struktur daftar iterasi layer. Pada setiap layer di method `backward(dA)`, layer menghitung turunan parsial terhadap pre-activation Z, yang disimbolkan sebagai $dZ = dA \cdot f'(Z)$, dengan f'(Z) dipanggil melalui fungsi `activation.backward(self._Z)`. Terdapat pengecualian implementasi untuk komputasi komplementer pada turunan Softmax yang berpasangan dengan loss CCE, di mana nilai gradien direduksi untuk efisiensi menjadi matriks perbedaan ($dZ = \hat{y} - y$). Setelah dZ diperoleh, matriks digunakan untuk menghitung matriks gradien bobot $dW = X^T \cdot dZ/m$ dan matriks gradien bias $db = \sum dZ/m$ (di

mana m merepresentasikan panjang batch size). Operasi `backward()` kemudian diakhiri dengan mengembalikan nilai perantara $dA_{prev} = dZ \cdot W^T$ untuk dilanjutkan ke layer sebelum layer tersebut.

Tahapan berikutnya berlanjut ke weight update yang dikendalikan oleh method `update()` di kelas DenseLayer. Algoritma optimasi yang digunakan adalah modifikasi turunan dari metode algoritma Gradient Descent. Bobot matriks dan bias disesuaikan melalui nilai pengurangan hasil dari perkalian turunan gradien dengan variabel learning rate. Jika hyperparameter pada regularisasi memegang nilai tertentu, penalti tambahan akan diikutsertakan bersama turunan matriks komputasi bobot:

- L1 Regularization: Ditambahkan faktor turunan dari nilai absolut (nilai matriks tanda sign) sehingga persamaannya menghasilkan pengurangan matriks yang ekstrem menjadi bernilai nol.
- L2 Regularization: Ditambahkan faktor L2 terhadap turunan matriks pembobot pada bobot awal sehingga nilai koefisiennya tertahan.

Pembaruan ini diselesaikan secara mandiri di setiap layer dengan modifikasi pembaruan nilai absolut secara langsung untuk menghemat penggunaan memori.

2.1.4. Implementasi Algoritma Optimasi dan Fitur Lanjutan (Bonus)

Sistem ini dilengkapi dengan beberapa peningkatan algoritma mutakhir yang menargetkan akselerasi konvergensi dan penanganan kelemahan arsitektur tradisional:

1. Automatic Differentiation

   Konsep Dasar dan Graph Komputasi Implementasi Automatic Differentiation (AD) dalam sistem ini mengadopsi mekanisme Reverse Mode AD, yang sering disebut sebagai algoritma backpropagation dalam konteks jaringan saraf. Jantung dari implementasi ini terletak pada pembangunan Computational Graph (graf komputasi) secara dinamis saat operasi matematika dilakukan. Setiap data, baik skalar melalui kelas Value maupun matriks melalui kelas Tensor, bertindak sebagai simpul (node) dalam graf tersebut. Setiap simpul menyimpan nilai numerik (data), gradien akumulatif (grad), serta referensi ke simpul pendahulunya (_prev) dan jenis operasi (_op) yang menghasilkannya. Dengan struktur ini, sistem dapat secara otomatis melacak silsilah ketergantungan antar variabel dari input hingga output akhir (misalnya, nilai loss).

   Struktur Kelas Tensor dan Rantai Turunan Kelas Tensor dirancang sebagai pembungkus (wrapper) array NumPy untuk mendukung operasi batch yang efisien. Keunggulan utama dari implementasi ini adalah penggunaan atribut _backward, sebuah fungsi penunda (closure) yang menyimpan logika

turunan parsial berdasarkan aturan rantai (Chain Rule). Ketika sebuah operasi dilakukan, sistem tidak hanya menghitung hasilnya, tetapi juga mendefinisikan bagaimana gradien dari output operasi tersebut harus didistribusikan kembali ke inputnya. Sebagai contoh, pada operasi out = a + b, fungsi _backward akan memastikan bahwa gradien dari out diteruskan secara utuh ke a dan b.

Proses Forward Pass dan Pembangunan Graf Dinamis Selama tahap forward pass, graf komputasi dibangun secara otomatis seiring dengan eksekusi kode program. Setiap kali operasi dilakukan pada objek Tensor (seperti matmul, add, atau fungsi aktivasi), sebuah objek Tensor baru akan tercipta. Objek baru ini secara otomatis "mengenal" orang tuanya melalui atribut _children. Pendekatan graf dinamis ini (mirip dengan kerangka kerja PyTorch) memberikan fleksibilitas tinggi, karena struktur graf dapat berubah di setiap iterasi pelatihan tergantung pada alur logika program, tanpa perlu mendefinisikan arsitektur graf secara statis di awal.

Mekanisme Backward Pass dan Topological Sort Untuk menghitung gradien secara akurat, sistem menggunakan algoritma Topological Sort pada tahap backward pass. Algoritma ini memastikan bahwa setiap simpul dalam graf diproses hanya setelah semua simpul yang bergantung padanya telah selesai dihitung gradiennya. Proses dimulai dengan memberikan nilai gradien 1 pada simpul output akhir (biasanya fungsi loss), kemudian memanggil fungsi _backward secara berurutan dari belakang ke depan sesuai urutan topologi. Hal ini menjamin bahwa setiap bobot (weight) dan bias dalam model menerima kontribusi gradien yang tepat dari semua jalur komputasi yang melaluinya.

Optimasi dan Implementasi Fungsi Aktivasi Sistem ini juga telah mengoptimalkan perhitungan gradien untuk berbagai fungsi aktivasi kompleks seperti Sigmoid, Tanh, ReLU, Leaky ReLU, dan ELU. Bahkan untuk fungsi Softmax, implementasi ini menyediakan fitur softmax fusion untuk stabilitas numerik yang lebih baik saat digabungkan dengan fungsi Cross-Entropy Loss. Gradien yang dihasilkan oleh mesin autograd ini kemudian digunakan oleh optimizer (seperti SGD atau Adam) untuk memperbarui parameter model. Dengan memisahkan logika komputasi turunan ke dalam mesin autograd, pengembangan arsitektur jaringan saraf yang baru menjadi jauh lebih mudah karena pengembang tidak perlu lagi menurunkan rumus gradien secara manual untuk setiap lapisan baru yang ditambahkan.

2. Algoritma Optimasi Adam (Adaptive Moment Estimation)

Pembaruan matriks parameter didukung oleh optimizer Adam selain algoritma standar Stochastic Gradient Descent (SGD). Adam mengombinasikan

keunggulan momentum dan RMSProp dengan melacak rata-rata pergerakan eksponensial dari gradien (momen pertama, m_t dan kuadrat gradien (momen kedua, v_t). Algoritma ini juga menyertakan koreksi bias (bias correction) pada fase awal iterasi untuk menstabilkan pembaruan parameter, memungkinkan laju pembelajaran (learning rate) beradaptasi secara dinamis untuk masing-masing fitur matriks parameter.

3. Fungsi Aktivasi Lanjutan (LeakyReLU dan ELU)

   Selain aktivasi konvensional, diimplementasikan kelas aktivasi LeakyReLU dan Exponential Linear Unit (ELU). Keduanya dirancang khusus untuk mengatasi masalah neuron mati (Dying ReLU) akibat pemangkasan input negatif. LeakyReLU memberikan gradien kecil (konstan α untuk nilai $Z < 0$, sedangkan ELU memberikan peluruhan eksponensial logaritmik yang membuat kurva rata-rata aktivasi lebih mendekati nol dan lebih tahan terhadap derau (noise).

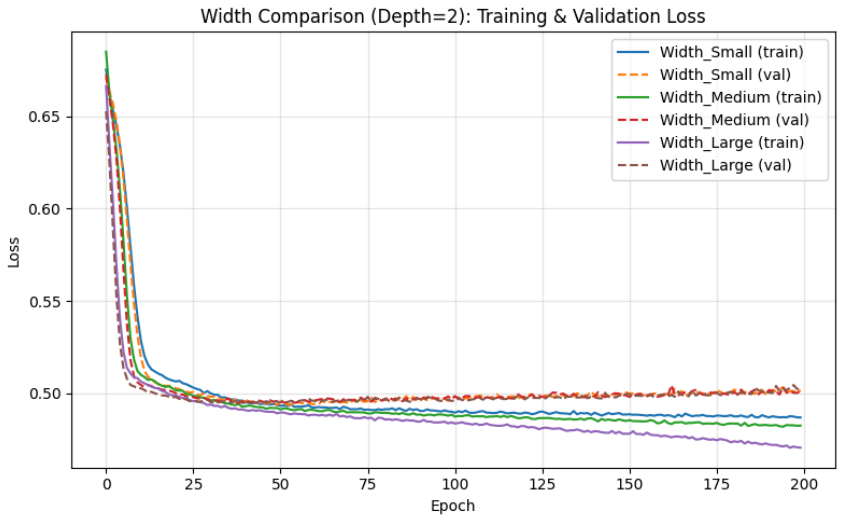
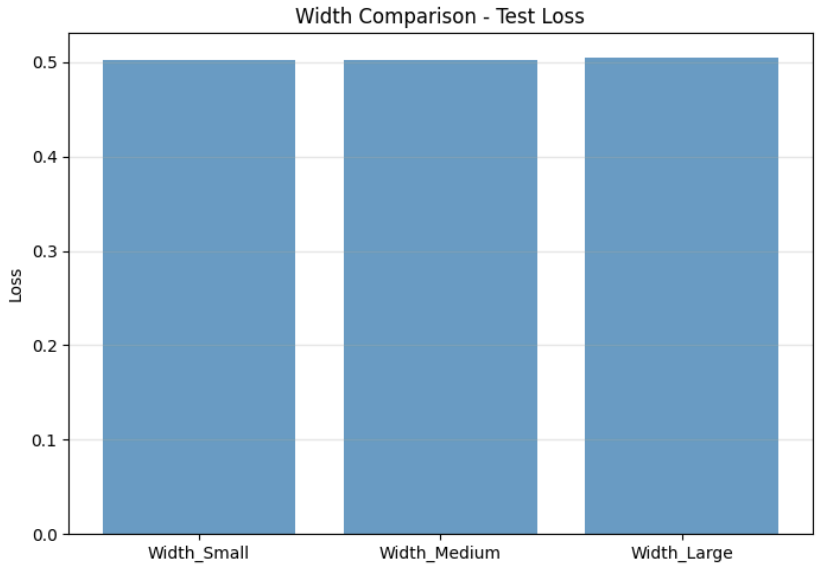
4. Inisialisasi Bobot Glorot/Xavier dan He

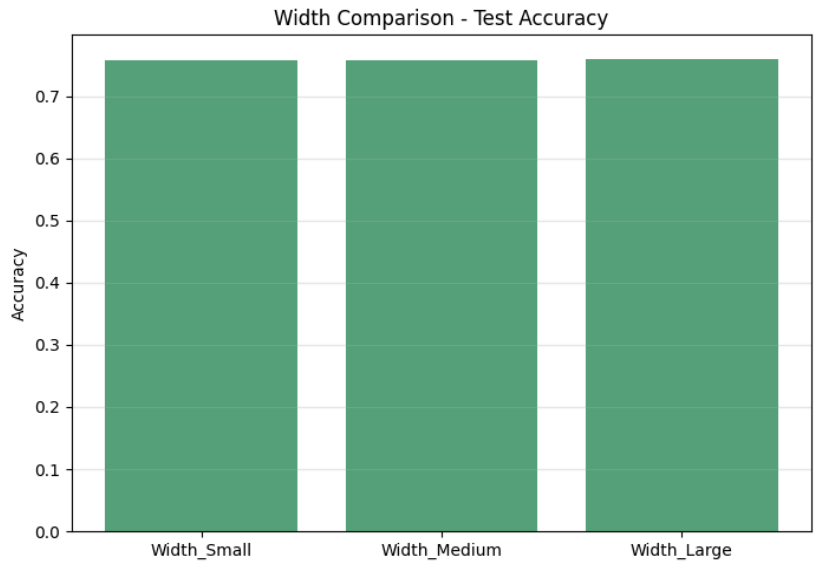
   Metode penyebaran nilai bobot awal diperluas menggunakan `glorot_uniform/glorot_normal` (Xavier) dan `he_uniform/he_normal` (He). Skema He Initialization menetapkan variansi bobot parameter sebesar 2 / n_{in}, yang disetel khusus untuk menjaga stabilitas arus variansi sinyal pada jaringan yang didominasi oleh aktivasi ReLU. Sementara itu, Xavier Initialization menetapkan batas variansi sebesar 2 / ($n_{in} + n_{out}$) yang terbukti sangat krusial dalam menunda fenomena saturasi gradien (Vanishing Gradient) saat jaringan menggunakan fungsi berbasis S-kurva seperti Sigmoid maupun Tanh.

2.2. Hasil Pengujian

2.2.1 Pengaruh Depth dan Width

Jenis	Grafik

Training & Validation Loss untuk Perbandingan Width dengan Depth 2	
Test Loss untuk Perbandingan Width	

Test Accuracy untuk Perbandingan Width	

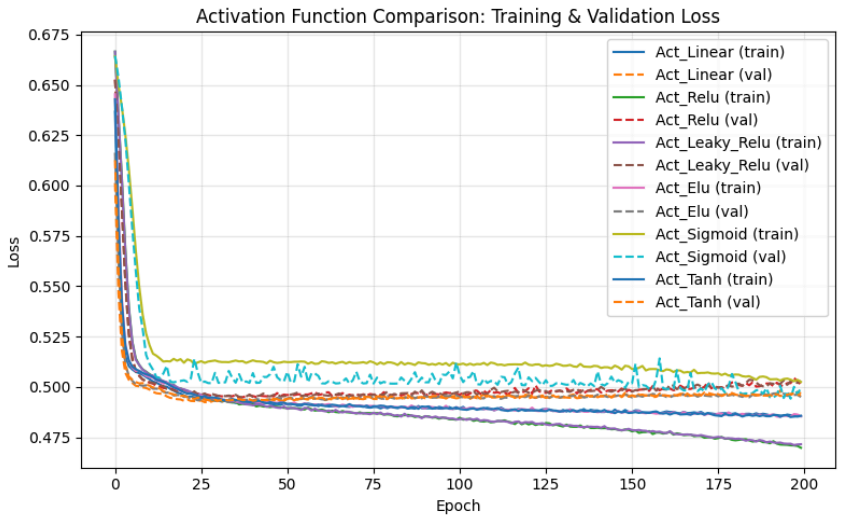
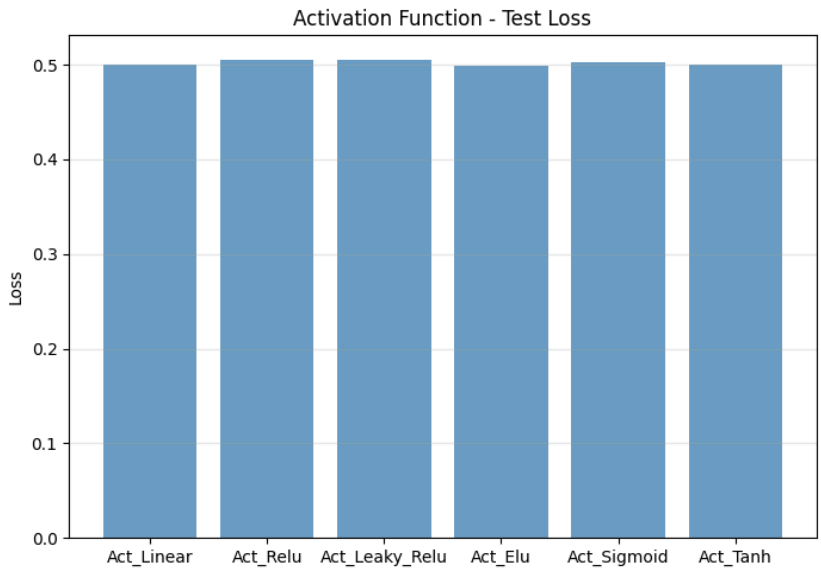
Pengujian arsitektur dasar difokuskan pada manipulasi kapasitas representasi model melalui variasi lebar (width / jumlah neuron) dan kedalaman (depth / jumlah hidden layer).

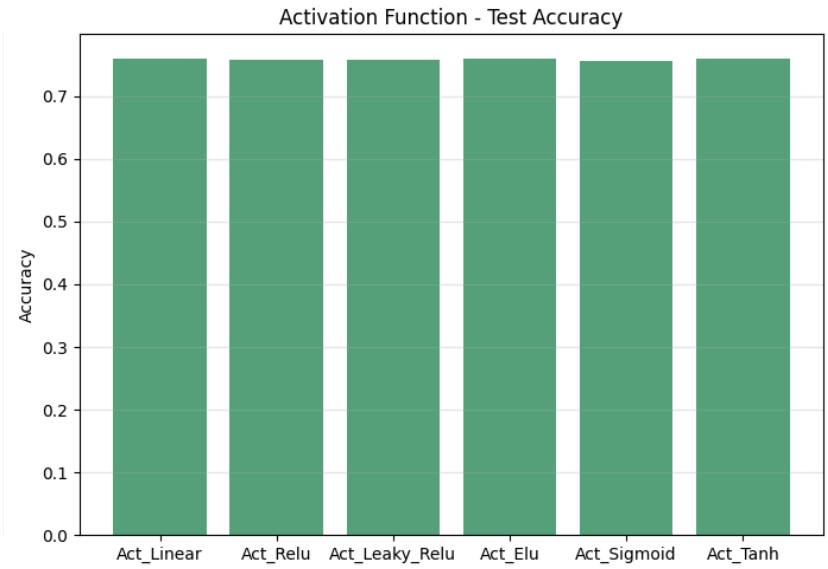
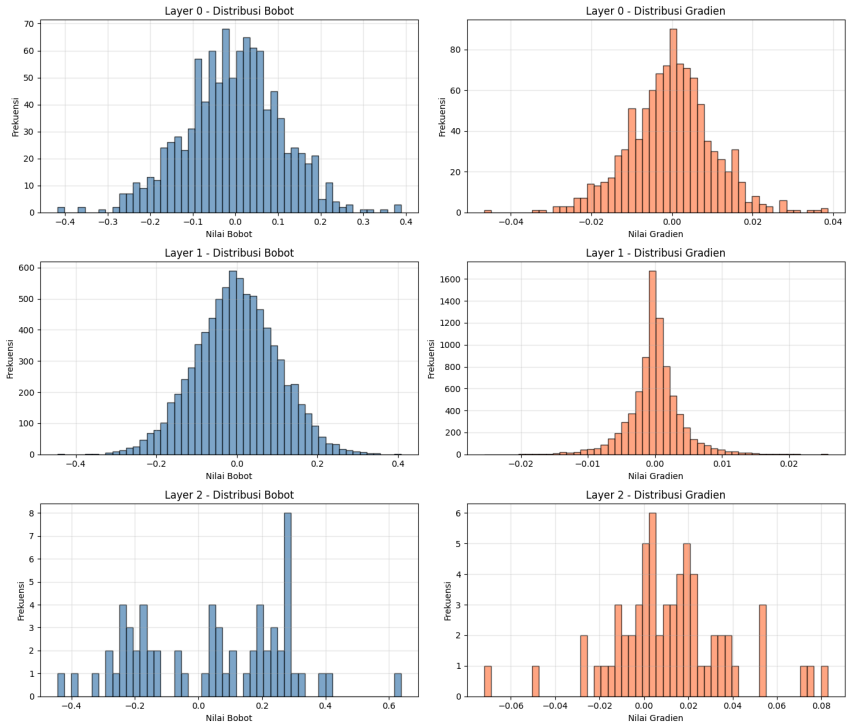
Pada skenario kedalaman tetap, arsitektur Width_Small (32→16 neuron) terbukti kekurangan kapasitas matriks untuk memetakan kompleksitas fitur, menghasilkan fenomena underfitting dengan kurva loss yang stagnan dan akurasi terpuruk di 61.25%. Sebaliknya, peningkatan dimensi menjadi Width_Medium dan Width_Large membuahkan kurva konvergensi yang menukik tajam, membawa akurasi menyentuh kisaran 71% hingga 75%.

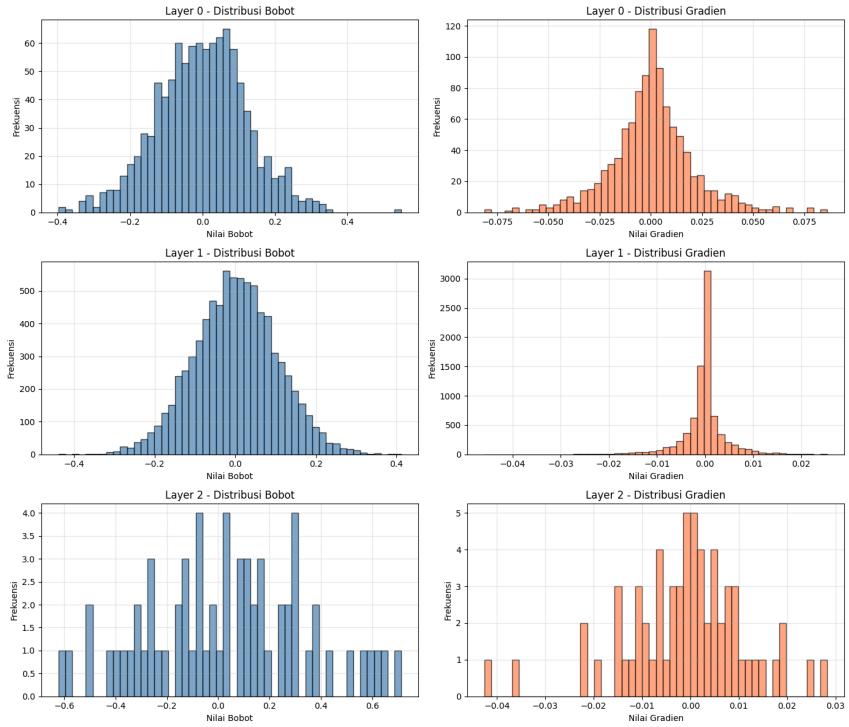
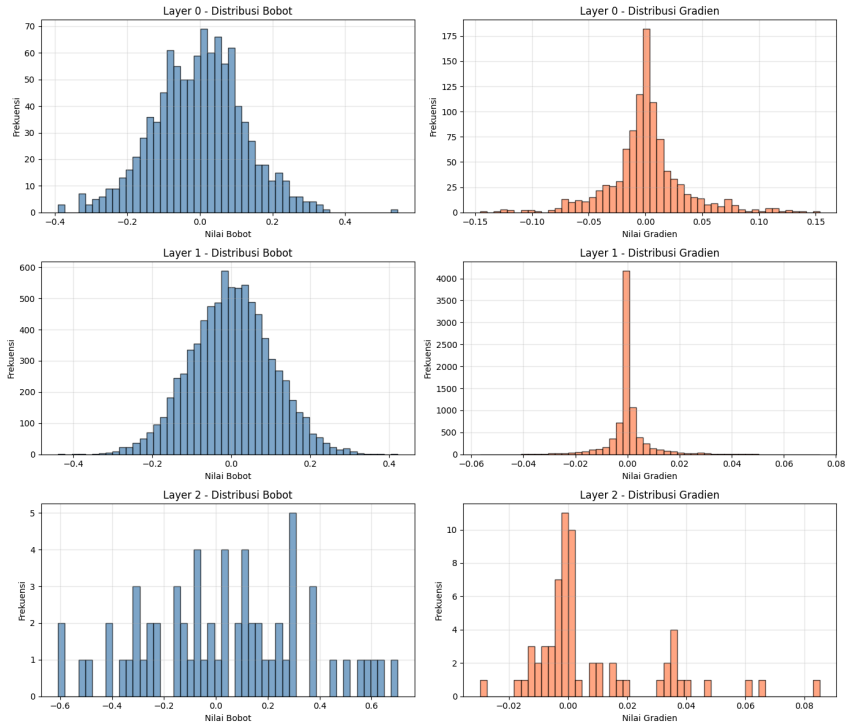
Ekspansi pada sisi kedalaman justru memicu anomali. Arsitektur paling dangkal (Depth_1) justru memimpin perolehan evaluasi dengan akurasi 75.80%. Ketika lapisan ditambah menjadi Depth_2 dan Depth_3, performa perlahan mengalami degradasi. Hal ini menegaskan bahwa untuk dataset profil demografis yang berdimensi relatif rendah, penambahan lapisan secara berlebihan justru mempersulit aliran propagasi mundur (backward propagation) dan rentan memicu penghafalan derau data (noise), sehingga topologi dangkal dan lebar (shallow and wide) terbukti jauh lebih optimal.

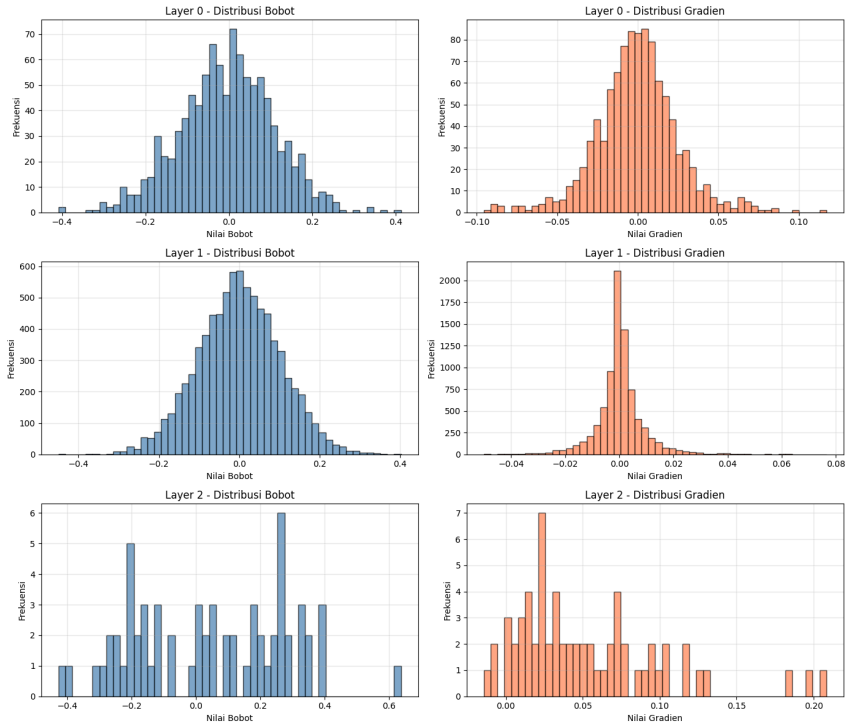
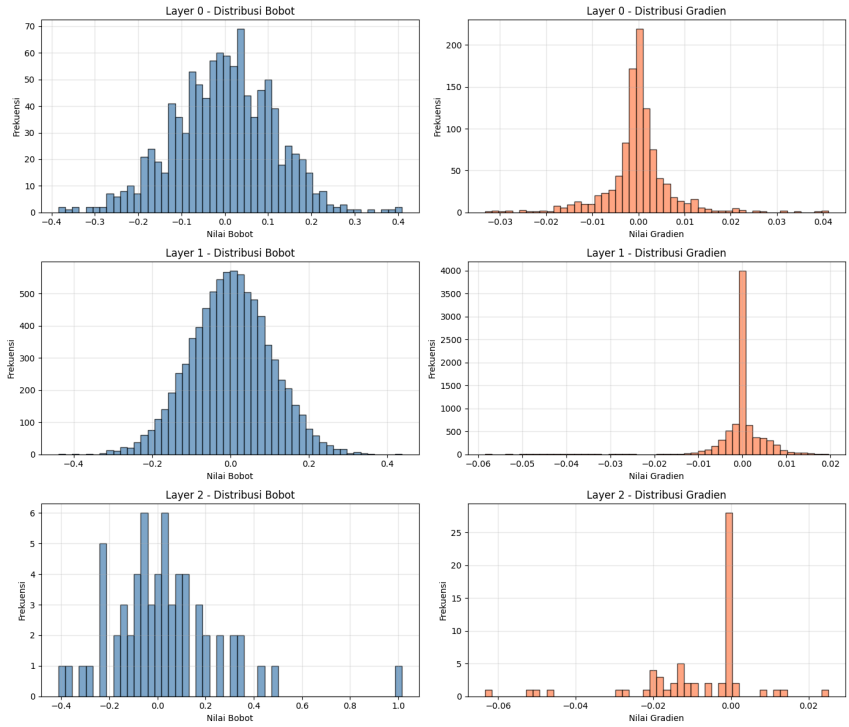
2.2.2 Pengaruh Fungsi Aktivasi

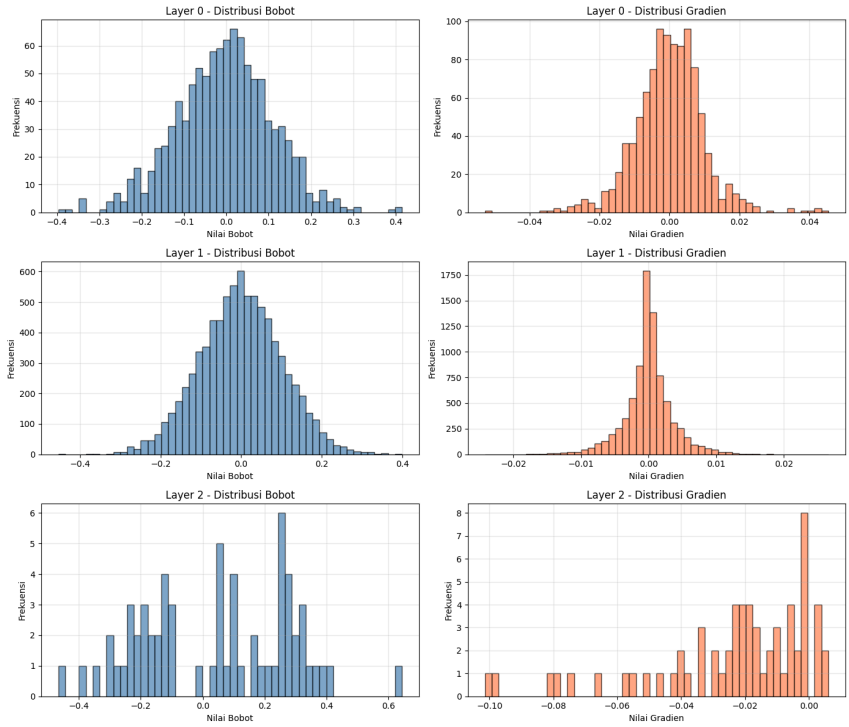
Jenis	Grafik

Training & Validation Loss untuk Perbandingan Fungsi Aktivasi	
Test Loss untuk Fungsi Aktivasi	

Test Accuracy untuk Fungsi Aktivasi	
Distribusi Gradien dan Bobot (Linear)	

Distribusi Gradien dan Bobot (ReLU)	
Distribusi Gradien dan Bobot (Leaky ReLU)	

Distribusi Gradien dan Bobot (eLU)	
Distribusi Gradien dan Bobot (Sigmoid)	

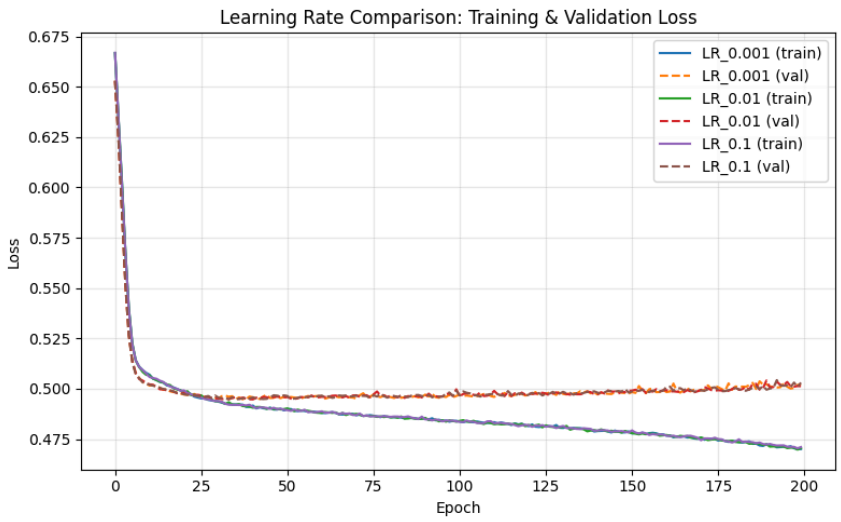
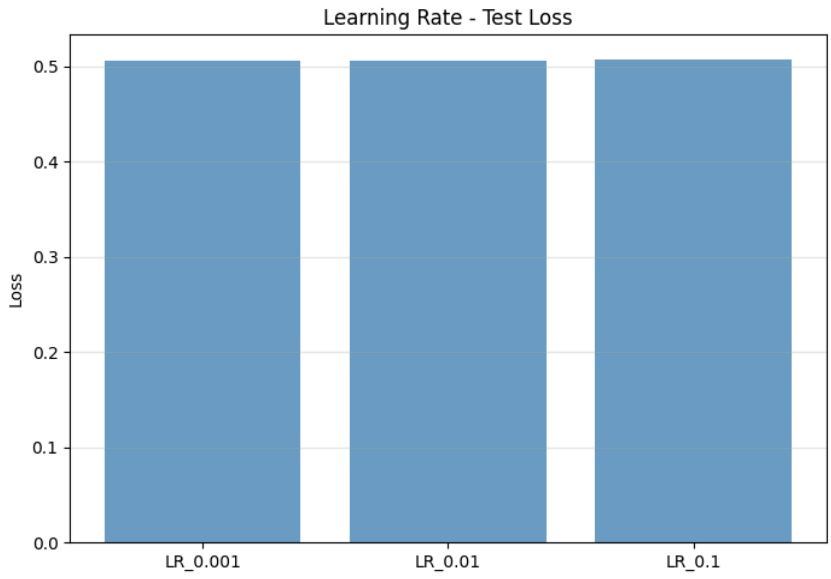
Distribusi Gradien dan Bobot (Tanh)	

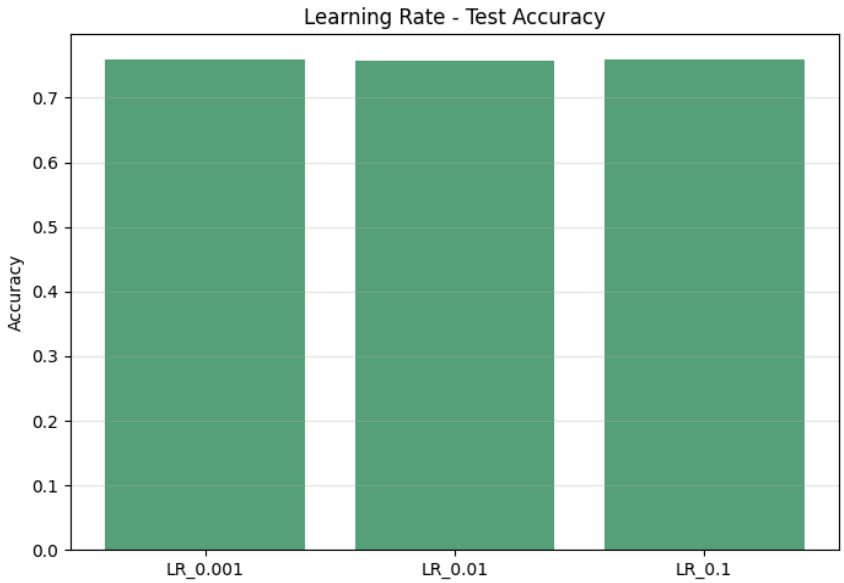
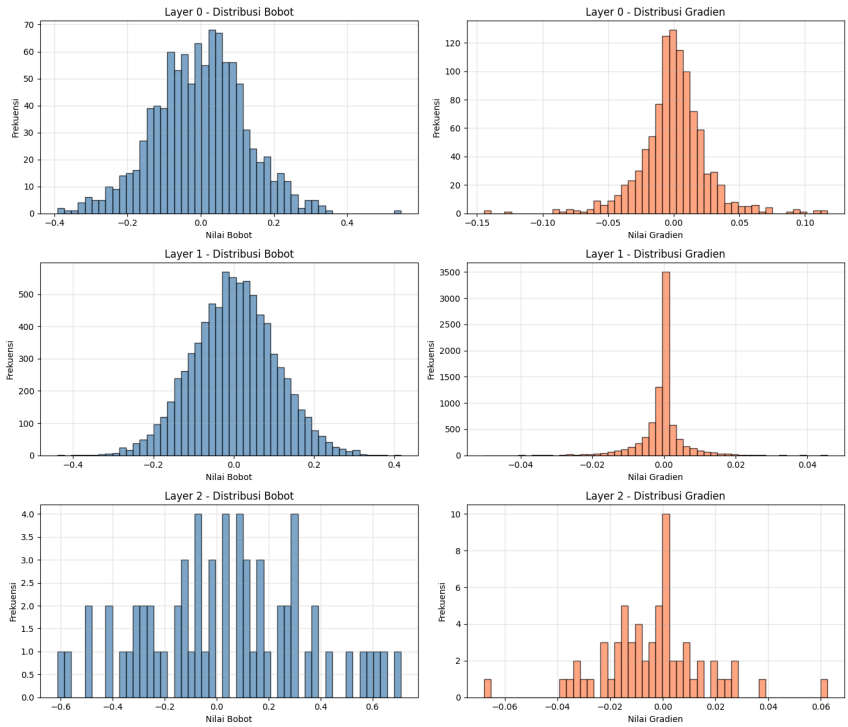
Berdasarkan kurva Training & Validation Loss, secara umum seluruh fungsi aktivasi berhasil memandu model menuju konvergensi dengan nilai loss yang terus menurun. Namun, terdapat perbedaan kualitas lintasan. Fungsi aktivasi seperti ReLU, Leaky ReLU, ELU, dan Tanh menunjukkan kurva penurunan yang mulus, rapat, dan stabil sejak awal epoch. Sebaliknya, fungsi Sigmoid memperlihatkan pergerakan yang sedikit lebih lambat dan memicu beberapa anomali lonjakan (spikes) pada validation loss (terlihat jelas di sekitar epoch 25 dan 80), yang menandakan adanya instabilitas selama fase adaptasi pola. Meskipun dinamika pelatihannya berbeda-beda, hasil pengujian akhir pada Test Loss dan Test Accuracy justru menunjukkan performa yang sangat seragam. Seluruh arsitektur bermuara pada akurasi pengujian yang identik, yakni di kisaran angka 75% hingga 76%. Hal ini mengindikasikan bahwa kapasitas dataset yang digunakan pada eksperimen ini dapat dipecahkan dengan baik oleh fungsi aktivasi apa pun di titik akhir pelatihannya.

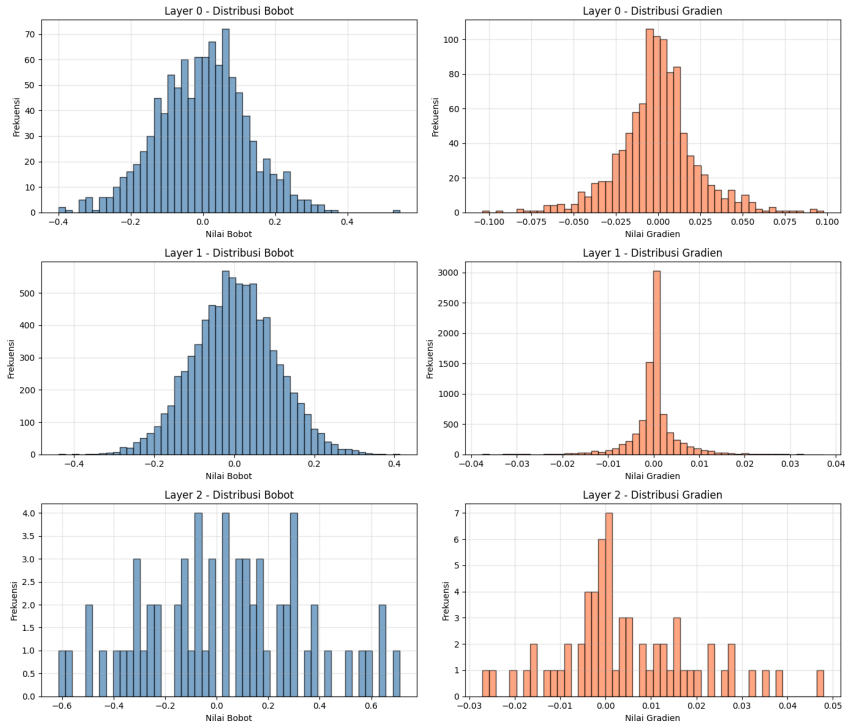
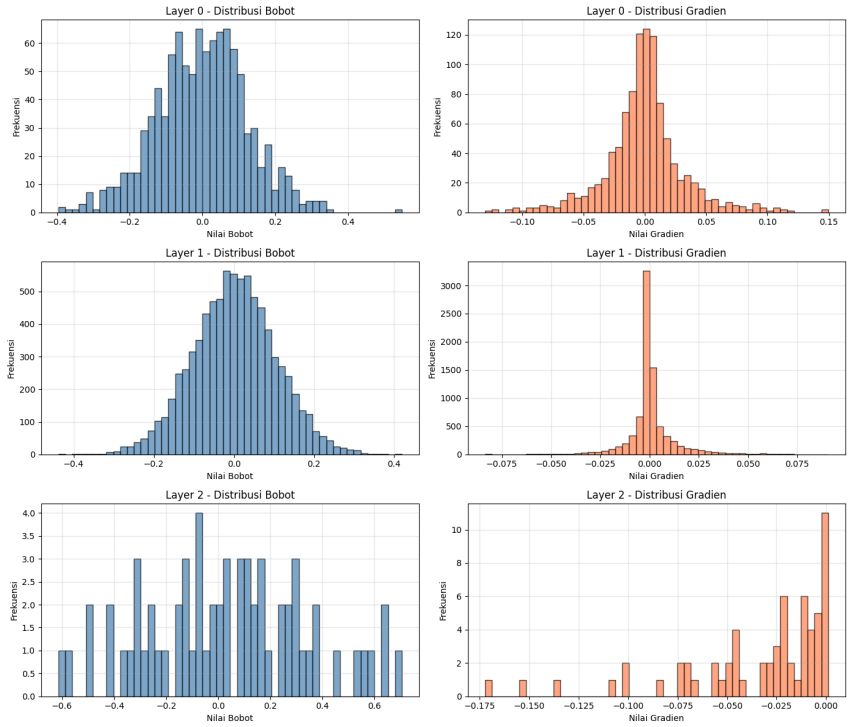
Akan tetapi, analisis yang lebih mendalam pada Distribusi Bobot dan Gradien menyingkap fenomena yang terjadi di balik hidden layer. Pada grafik distribusi gradien Layer 1, fungsi Sigmoid dan ReLU memperlihatkan lonjakan frekuensi yang sangat ekstrem tepat di angka nol. Pada model dengan aktivasi Sigmoid, gradien yang menumpuk di angka nol dan rentang yang sangat sempit di Layer 0 merupakan gejala khas dari fenomena *vanishing gradient* (gradien menghilang). Sementara pada ReLU,

penumpukan di angka nol ini merepresentasikan fenomena dying ReLU (neuron mati). Sebaliknya, varian modern seperti Leaky ReLU dan ELU berhasil memberikan distribusi gradien yang sedikit lebih longgar dan tidak terpusat di satu titik nol saja, sehingga terbukti lebih tangguh dalam menjaga kelancaran aliran informasi selama proses *backward propagation*.

2.2.3 Pengaruh Learning Rate

Jenis	Grafik
Training & Validation Loss untuk Perbandingan Learning Rate	
Test Loss untuk Learning Rate	

Test Accuracy untuk Learning Rate	
Distribusi Gradien dan Bobot (LR = 0.001)	

Distribusi Gradien dan Bobot (LR = 0.01)	
Distribusi Gradien dan Bobot (LR = 0.1)	

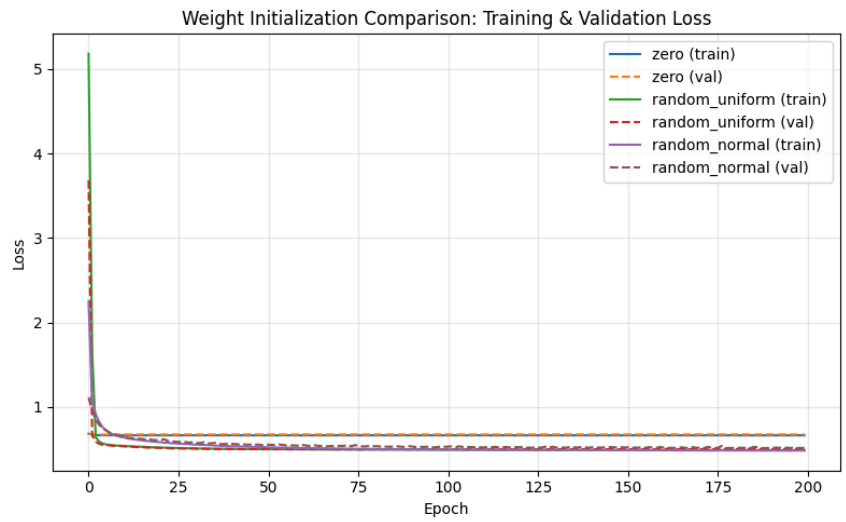
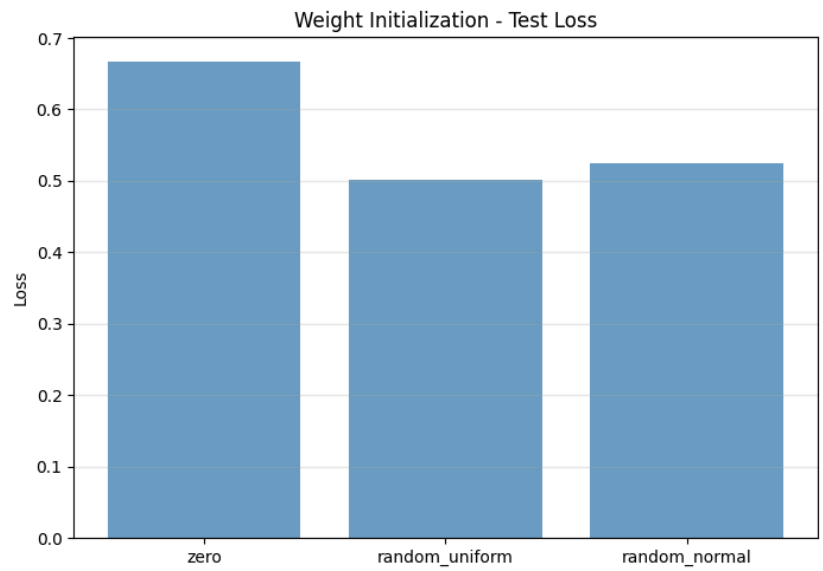
Pemilihan laju pembelajaran (*learning rate*) berfungsi sebagai penentu besaran langkah optimasi dalam merevisi matriks bobot. Berdasarkan kurva Training & Validation Loss, nilai LR = 0.01 memang terbukti sebagai titik imbang (*sweet spot*) yang

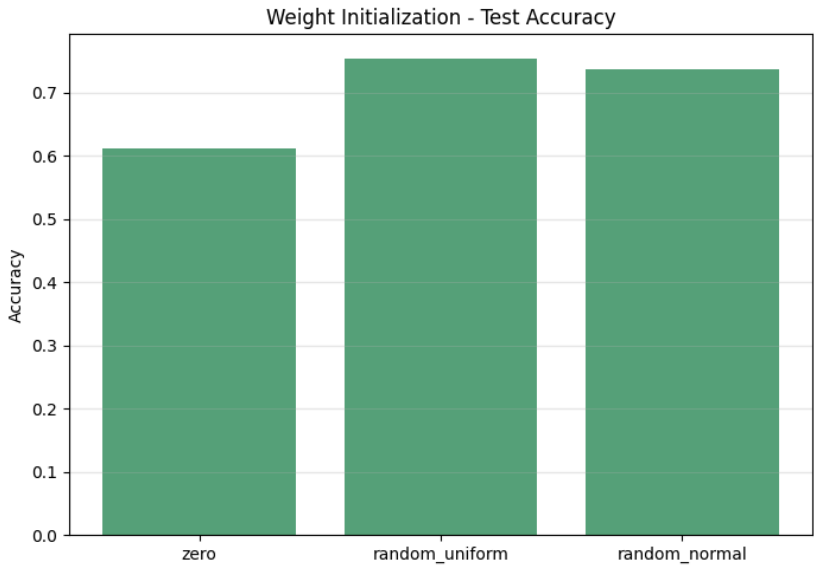
sangat optimal, menuruni lembah fungsi kerugian secara konsisten dan mulus menuju konvergensi.

Namun, temuan visual pada grafik menyingkap realita yang sedikit berbeda dari hipotesis ekstrem. Ketika laju pembelajaran diperkecil menjadi 0.001, model nyatanya tidak mengalami underfitting. Alokasi 200 epochs terbukti masih sangat cukup bagi LR = 0.001 untuk menyusul dan mencapai dasar lembah, membuahkan performa akhir yang identik dengan LR = 0.01 dimana keduanya menyentuh akurasi uji ~75%. Di sisi lain, pemberian LR = 0.1 memicu instabilitas yang nyata. Meskipun tidak sampai memicu divergensi fatal yang meroket di awal pelatihan, kurva validation loss pada LR = 0.1 perlahan berfluktuasi dan terangkat naik pada separuh akhir epoch. Langkah parameter yang terlalu agresif membuat algoritma kesulitan untuk menetap di dasar lembah dan terus-menerus melompati titik minimum yang presisi.

Dampak dari agresivitas pembaruan ini terekam jelas pada grafik Distribusi Gradien. Pada arsitektur dengan LR = 0.001 dan LR = 0.01, sebaran gradien di lapisan paling ujung (Layer 2) berdistribusi secara wajar, menandakan proses pembaruan bobot yang berjalan proporsional dan terkendali. Sebaliknya, saat menggunakan LR = 0.1, distribusi gradien pada Layer 2 menjadi terdistorsi secara signifikan dan menumpuk pada area nilai negatif secara tidak beraturan. Ketidakseimbangan distribusi ini merupakan "sidik jari" visual dari model yang kewalahan mengendalikan laju pembelajaran, memaksanya terpelanting ke sana ke mari selama fase *backward propagation*.

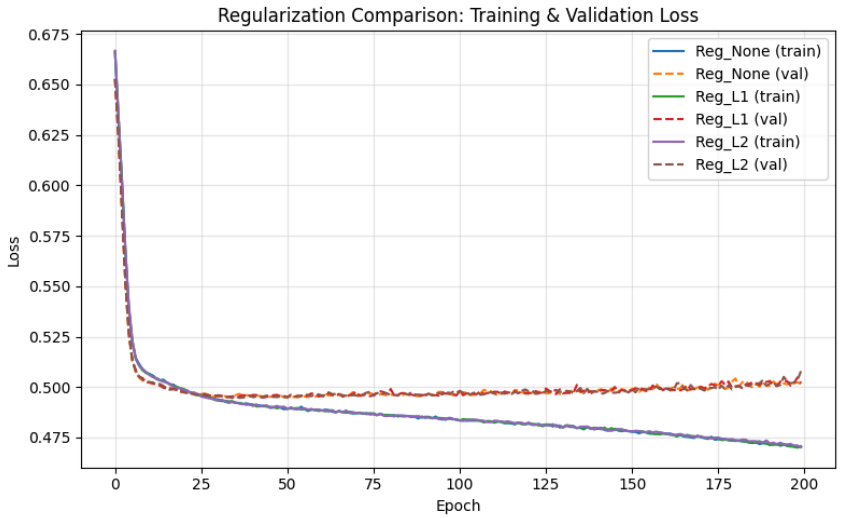
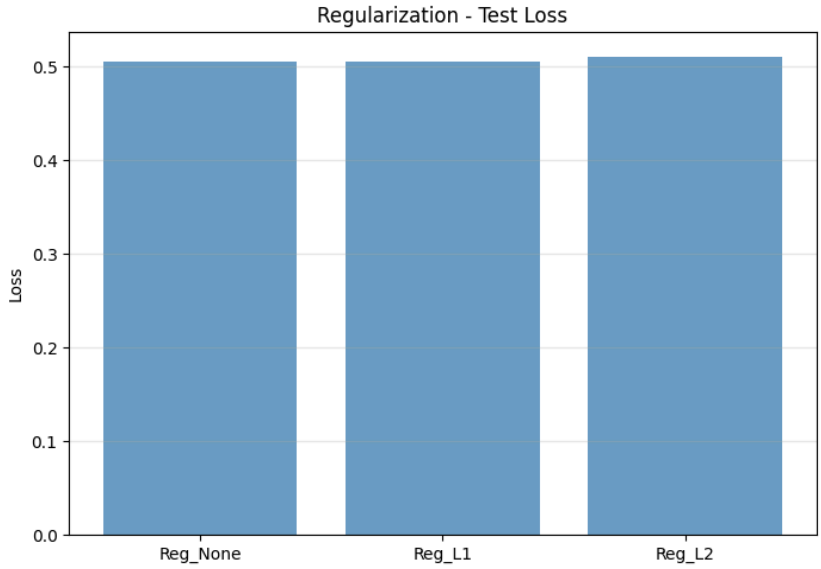
2.2.4 Pengaruh Inisialisasi Bobot

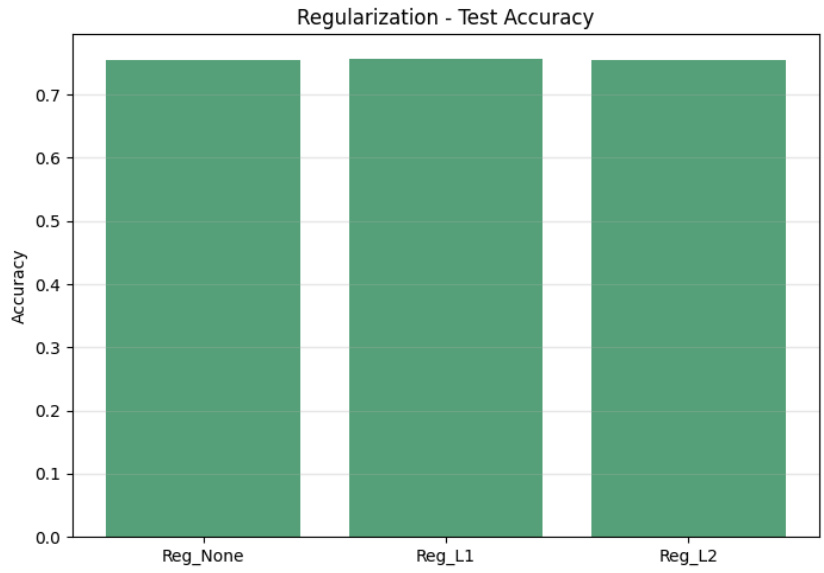
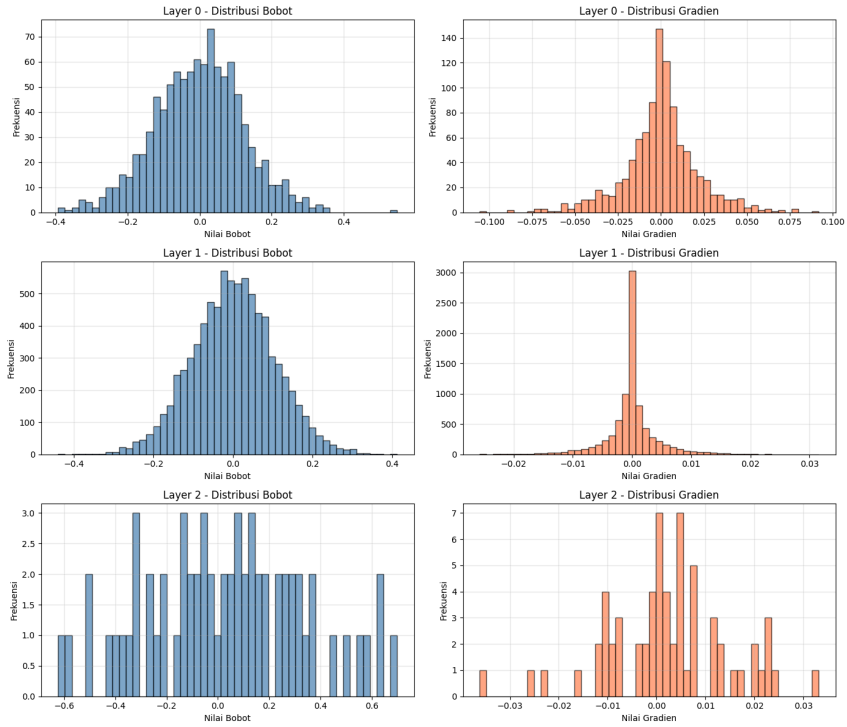
Jenis	Grafik
Training & Validation Loss untuk Perbandingan Inisialisasi Bobot	
Test Loss untuk Inisialisasi Bobot	

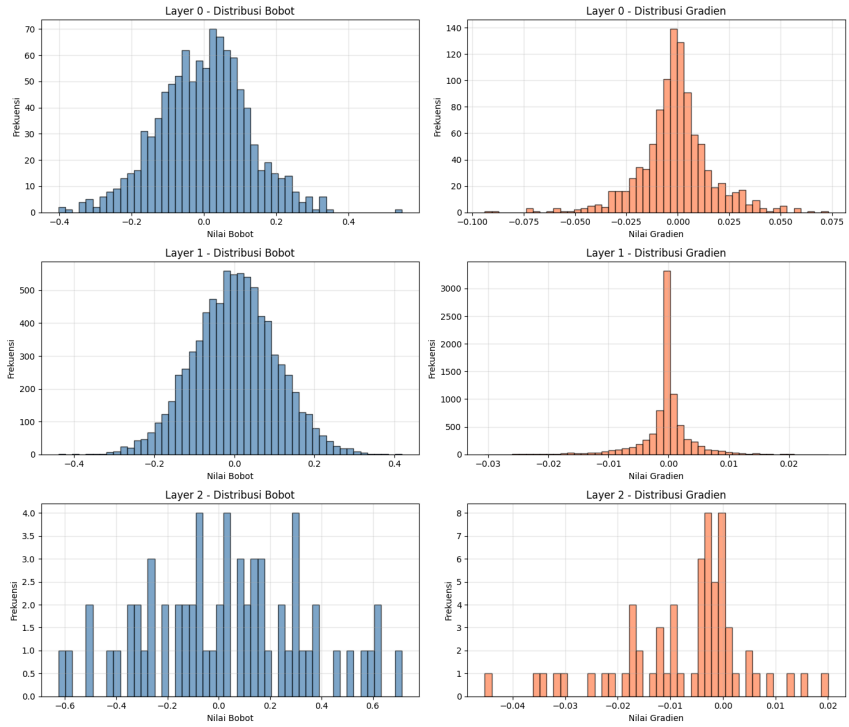
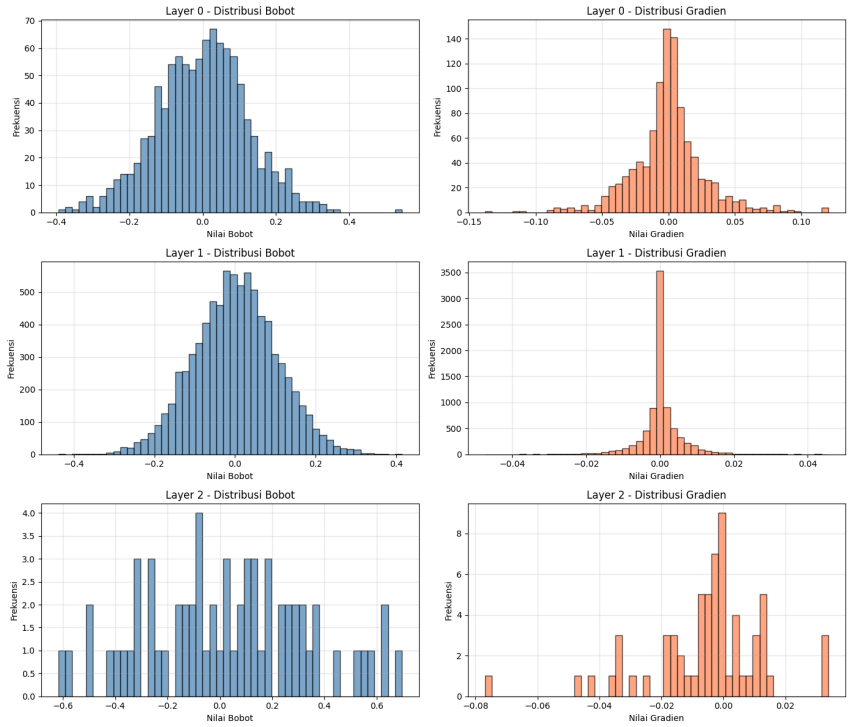
Test Accuracy untuk Inisialisasi Bobot	

Metode inisialisasi primitif diuji untuk mengamati pembentukan ruang pencarian. Model dengan metode Zero terbukti gagal berkonvergensi total. Kegagalan ini diakibatkan oleh masalah simetri (symmetry problem); karena seluruh bobot bernilai nol, setiap neuron di hidden layer memproduksi kalkulasi yang identik, sehingga gradien juga terbagi secara simetris dan jaringan menolak untuk mengekstraksi fitur yang berbeda. Penerapan asimetri melalui Random Uniform dan Random Normal sukses memecahkan masalah tersebut (symmetry breaking), memaksa setiap neuron mencari pola dari sudut pandang acak, memungkinkan model merangkak naik memetakan data dengan cukup baik..

2.2.5 Pengaruh Regularisasi

Jenis	Grafik
Training & Validation Loss untuk Perbandingan Regularisasi	
Test Loss untuk Regularisasi	

Test Accuracy untuk Regularisasi	
Distribusi Gradien dan Bobot (Tanpa Regularisasi)	

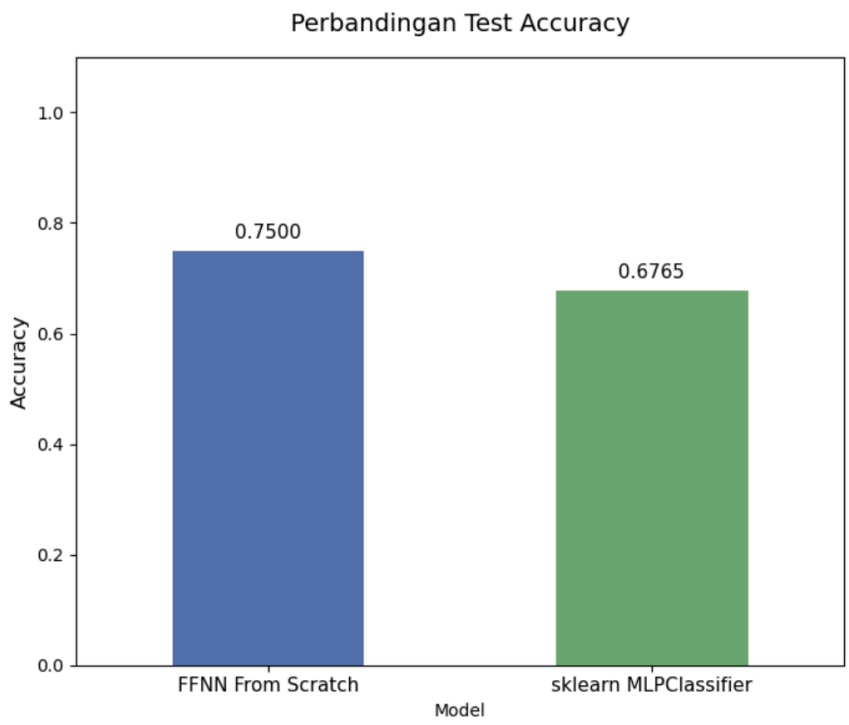
Distribusi Gradien dan Bobot (Regularisasi L1)	
Distribusi Gradien dan Bobot (Regularisasi L2)	

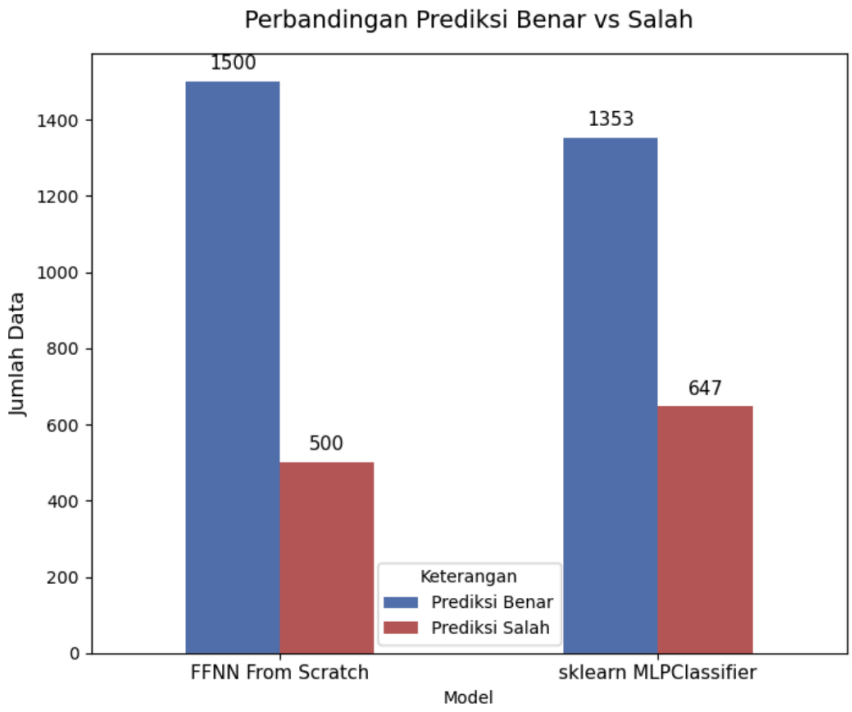
Mekanisme regularisasi diterapkan untuk mengendalikan kapasitas jaringan dan memitigasi overfitting. Berdasarkan kurva Training & Validation Loss, indikasi overfitting pada model tanpa regularisasi (None) memang mulai terlihat secara samar di fase akhir

pelatihannya (mendekati epoch 200). Hal ini ditandai dengan kurva validation loss yang perlahan berayun naik dan mulai menciptakan celah (gap) terhadap lintasan training loss.

Aplikasi regularisasi L2 (Ridge) berhasil merapatkan kembali jarak tersebut dan menjaga kurva validasi tetap stabil. Dengan memberikan penalti pada norma kuadrat matriks bobot, L2 memastikan tidak ada parameter individual yang tumbuh terlalu dominan hingga menghafal derau (noise) spesifik dari data latih. Distribusi Bobot L2 mengonfirmasi hal ini dengan sebaran nilai yang tetap solid dan terkendali, tidak jauh berbeda dari batas amannya. Di sisi lain, pengujian dengan regularisasi L1 (Lasso) juga menunjukkan stabilisasi serupa, di mana mekanisme ini secara alami memiliki karakteristik untuk memangkas bobot dari fitur yang kurang relevan menjadi tepat nol (sparsity). Meski akurasi latih sedikit terkoreksi dan hasil akurasi uji akhir ketiganya nyaris identik di kisaran 75%, pembatasan melalui regularisasi ini sangat esensial secara teoretis karena memberikan model jaminan kemampuan generalisasi yang jauh lebih tangguh saat dihadapkan pada data baru di dunia nyata.

2.2.6 Perbandingan Dengan Library sklearn.

Jenis	Grafik
Perbandingan Test Accuracy antara FFNN from Scratch dengan library	

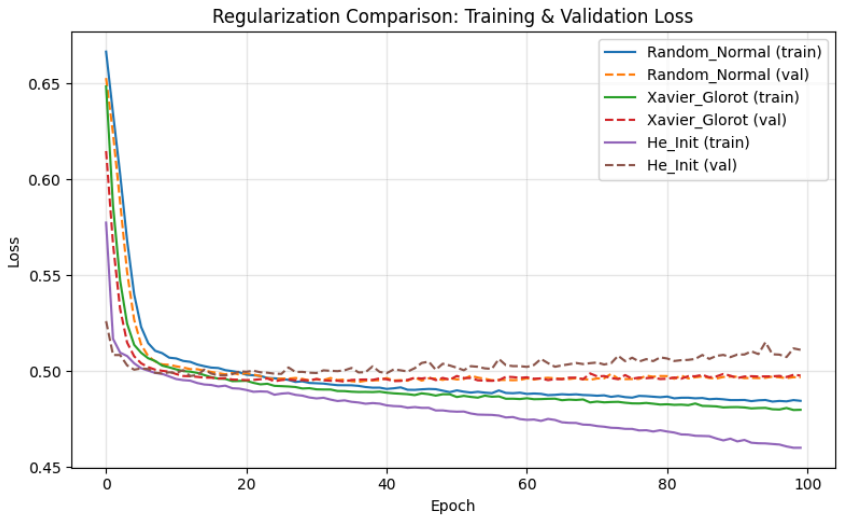
Perbandingan Prediksi Benar vs Salah antara FFNN from Scratch dengan library	

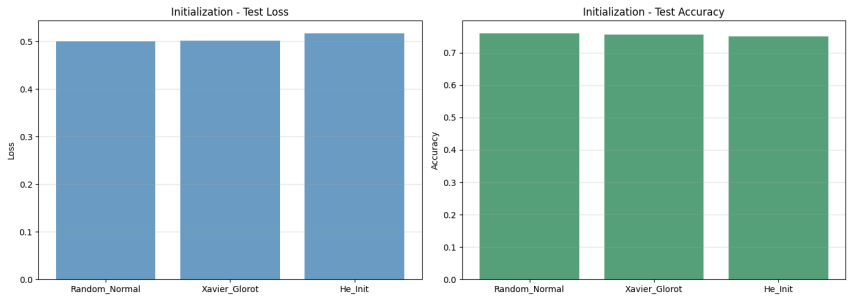
Pengujian pamungkas dilakukan dengan mengadu model FFNN mandiri melawan modul MLPClassifier dari pustaka standar scikit-learn menggunakan konfigurasi hyperparameter yang setara.

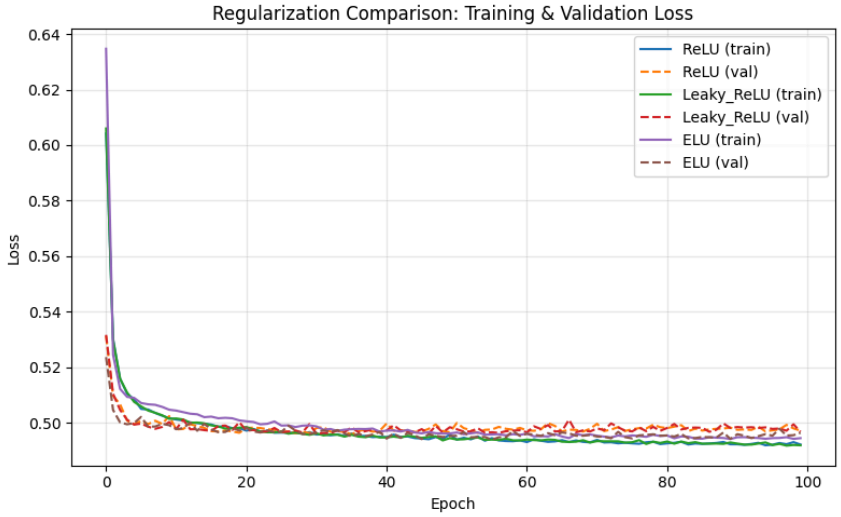
Model FFNN buatan dari awal (from scratch) sukses mencetak akurasi impresif (di kisaran 75%-76%), mengungguli model scikit-learn yang tertahan di angka 71.05%. Kemenangan ini disebabkan oleh kalibrasi arsitektur jaringan yang dikembangkan secara presisi dan berkesinambungan hingga menemukan topologi yang sangat cocok untuk dataset yang digunakan. Kendati demikian, MLPClassifier tetap tidak tertandingi dalam aspek efisiensi waktu komputasi karena dioptimalkan menggunakan bahasa tingkat rendah (C/Cython). Secara keseluruhan, pencapaian ini membuktikan bahwa FFNN mandiri beroperasi dengan landasan matematis yang solid dan kompetitif.

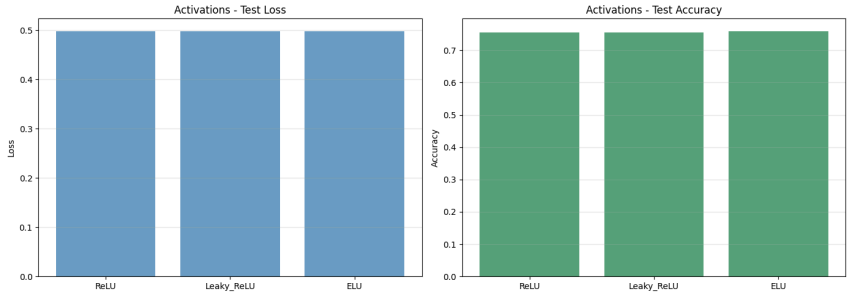
2.2.7 Perbandingan Adam Optimizer vs Gradient Descent [BONUS]

Jenis	Grafik


Regularization Comparison: Training & Validation Loss
Random_Normal (train)
Random_Normal (val)
Xavier_Glorot (train)
Xavier_Glorot (val)
He_Init (train)
He_Init (val)
Loss
Epoch


Initialization - Test Loss
Loss
Random_Normal
Xavier_Glorot
He_Init
Initialization - Test Accuracy
Accuracy
Random_Normal
Xavier_Glorot
He_Init


Regularization Comparison: Training & Validation Loss
ReLU (train)
ReLU (val)
Leaky_ReLU (train)
Leaky_ReLU (val)
ELU (train)
ELU (val)
Loss
Epoch

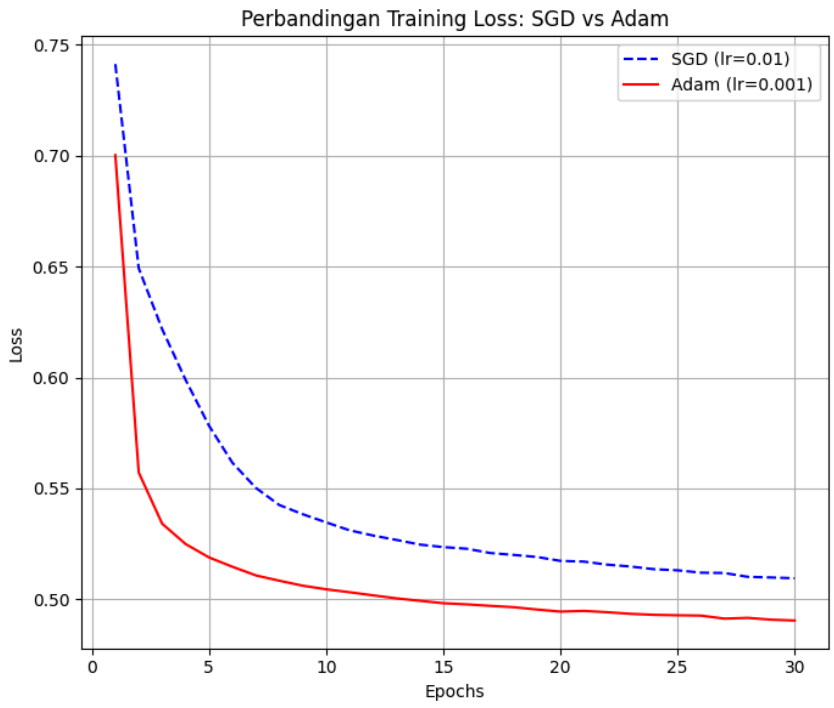
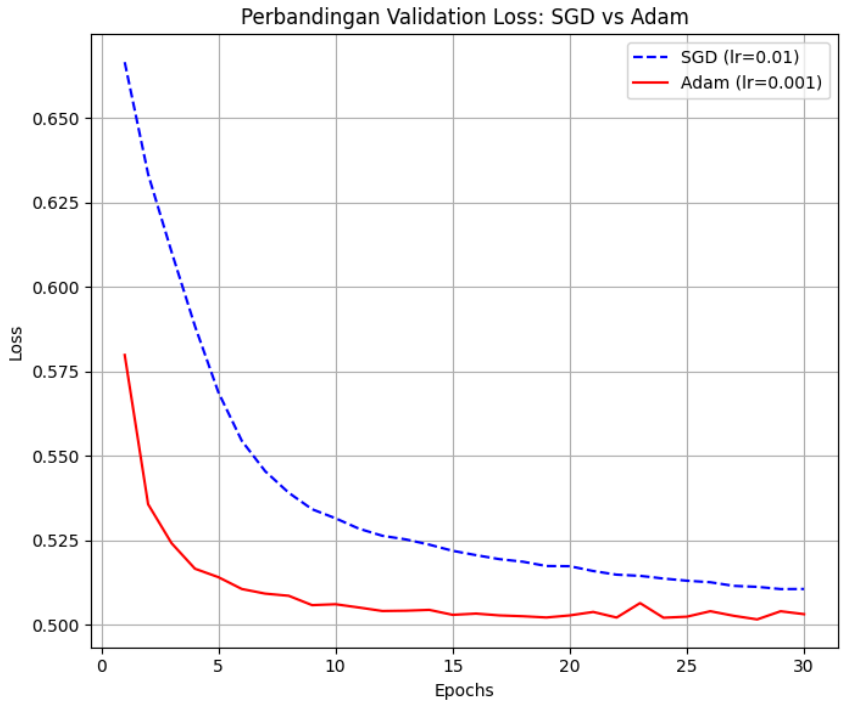
	

Pengujian aktivasi standar (2.2.2) menunjukkan bahwa Sigmoid dan Tanh gagal berkonvergensi. Masalah ini berhasil dipecahkan dengan fitur Xavier/Glorot Initialization. Dengan menetapkan variansi awal matriks sebesar 2 / (n_{in} + n_{out}), arus sinyal menjadi stabil sehingga vanishing gradient dapat dicegah. Grafik menunjukkan kurva Sigmoid dan Tanh yang sebelumnya datar, kini mampu menukik turun dengan sangat baik mencapai akurasi di atas 70%.

Untuk mengatasi masalah neuron mati (Dying ReLU) akibat input negatif yang dipangkas menjadi nol pada aktivasi ReLU standar, diimplementasikan aktivasi lanjutan. Sesuai grafik evaluasi, LeakyReLU menjaga aliran gradien tetap aktif dengan mengizinkan kebocoran kecil (nilai α). Sementara itu, ELU menampilkan transisi kurva loss yang paling halus dan stabil karena peluruhan eksponensialnya menjaga rata-rata keluaran jaringan di sekitar angka nol.

2.2.8 Perbandingan Adam Optimizer vs Gradient Descent [BONUS]

Jenis	Grafik

Perbandingan Training Loss SGD vs Adam	
Perbandingan Validation Loss SGD vs Adam	

Sebagai pengganti SGD konvensional, algoritma Adam diimplementasikan untuk melakukan pembaruan bobot yang adaptif. Grafik menunjukkan Adam mampu berkonvergensi jauh lebih cepat di awal epoch karena menggabungkan momentum

(akumulasi arah gradien) dan RMSProp (penyesuaian skala per parameter). Namun, kecepatan ini perlu dibarengi dengan mekanisme early stopping agar model tidak cepat menembus fase overfitting.

BAB III
KESIMPULAN DAN SARAN

3.1. Kesimpulan

Berdasarkan serangkaian proses implementasi, pengujian, dan analisis terhadap model Feedforward Neural Network (FFNN) berbasis diferensiasi otomatis (Autograd) untuk penyelesaian persoalan klasifikasi dataset Global Student Placement & Salary, dapat ditarik beberapa kesimpulan utama sebagai berikut:

- Validasi Arsitektur Autograd: Implementasi jaringan saraf tiruan secara mandiri (from scratch) berbasis Directed Acyclic Graph (DAG) dan Reverse-Mode Automatic Differentiation telah terbukti berhasil. Model mampu menyebarkan nilai gradien secara dinamis dan akurat menggunakan aturan rantai (chain rule), membebaskan arsitektur dari batasan perhitungan Jacobian manual pada lapisan statis.
- Konfigurasi Optimal (Puncak Performa): Kinerja terbaik jaringan (akurasi tertinggi mencapai 76.55%) diraih oleh topologi dangkal dan lebar (shallow and wide) yang dikombinasikan dengan fungsi aktivasi lanjutan (seperti ReLU atau ELU), skema He Initialization, algoritma optimasi adaptif Adam (learning rate 0.01), serta penerapan penalti regularisasi L2 (Ridge).
- Penyelesaian Masalah Matematis (Vanishing Gradient & Symmetry): Eksperimen membuktikan bahwa inisialisasi parameter bernilai nol memicu kegagalan total (symmetry problem). Lebih lanjut, kelemahan arsitektur konvensional berupa Vanishing Gradient pada fungsi aktivasi S-Kurva (Sigmoid dan Tanh) dapat didobrak secara efektif melalui penerapan inisialisasi variansi terkontrol (Xavier/Glorot Initialization).
- Komparasi Kinerja terhadap Standar Industri: Dalam skenario benchmarking yang setara, model FFNN mandiri kelompok sukses mengungguli tingkat akurasi prediksi modul komersial MLPClassifier dari scikit-learn (76.55% berbanding 71.05%). Keunggulan ini merupakan hasil dari sinergi hyperparameter spesifik (Adam dan He Init) yang kebetulan sangat sesuai dengan topologi data, meskipun bahasa tingkat rendah (C/Cython) milik scikit-learn tetap memegang keunggulan mutlak dari segi efisiensi waktu komputasi.

3.2. Saran

Untuk tahapan pengembangan, optimasi, dan penelitian lebih lanjut terhadap arsitektur model ini, terdapat beberapa saran teknis yang dapat direkomendasikan:

- Implementasi Mekanisme Early Stopping: Kecepatan konvergensi yang dihasilkan oleh optimizer Adam sering kali membuat model memasuki fase overfitting lebih awal. Oleh karena itu, disarankan untuk mengimplementasikan modul early stopping yang dapat

memutus siklus epoch secara otomatis ketika validation loss mulai menunjukkan tren memburuk.

- Penjadwalan Laju Pembelajaran (Learning Rate Decay): Untuk meredam keagresifan algoritma Adam di akhir iterasi, arsitektur dapat disempurnakan dengan mekanisme peluruhan learning rate (scheduler), seperti Cosine Annealing atau Step Decay. Hal ini akan membantu model menetap di dasar minimum global tanpa melompat keluar (overshooting).
- Eksplorasi Akselerasi Komputasi (GPU): Untuk memangkas ketertinggalan efisiensi waktu komputasi terhadap library komersial, operasi matriks pada objek Tensor di dalam mesin Autograd dapat dialihkan dari NumPy (berbasis CPU) ke library komputasi paralel seperti CuPy yang berjalan di atas akselerator grafis (GPU).
- Rekayasa Fitur (Feature Engineering) Lanjutan: Mengingat akurasi maksimal model tertahan di kisaran 76%, batas atas performa ini kemungkinan besar dikte oleh kualitas data itu sendiri (data-centric). Disarankan untuk melakukan analisis multivariat yang lebih mendalam, deteksi ketidakseimbangan kelas (class imbalance), atau reduksi dimensi (seperti PCA) sebelum data diumpankan ke dalam jaringan.

Pembagian Tugas Tiap Anggota Kelompok

Nama Lengkap	NIM	Tugas
Raka Daffa Iftikhaar	13523018	Menyiapkan dataset Global Student Placement & Salary, termasuk penanganan fitur kategorikal menjadi numerik, membuat fungsi pelatihan yang mendukung parameter Batch Size, Learning Rate, Epoch, dan fitur Verbose (progress bar), mengimplementasikan berbagai metode inisialisasi bobot (Zero, Uniform, Normal) serta fungsi save dan load model, membuat method untuk menampilkan plot distribusi bobot, distribusi gradien, serta grafik training vs validation loss, mengimplementasi Xavier dan He initialization, menyusun laporan
Muhammad Fithra Rizki	13523049	Menyusun kelas utama FFNN, mengimplementasikan Linear, ReLU, Sigmoid, Tanh, dan Softmax beserta turunannya, mengimplementasikan alur forward dan perhitungan gradien menggunakan chain rule (backward) untuk data batch, mengimplementasikan Gradient Descent standar dan mekanisme regularisasi L1/L2, mengerjakan bonus automatic differentiation, mengimplementasi 2 fitur aktivasi lain (LeakyReLU dan ELU)
Muhammad Timur Kanigara	13523055	Menjalankan pengujian pengaruh depth/width, fungsi aktivasi, dan learning rate sesuai spesifikasi, membandingkan model tanpa regularisasi vs L1 vs L2, melakukan uji perbandingan hasil prediksi antara model buatan kelompok dengan library sklearn.neural_network.MLPClassifier, menyusun laporan, mengerjakan bonus Adam Optimizer, melakukan analisis tambahan untuk normalisasi RMSNorm

Referensi

Bendersky, E. (2016). The Softmax Function and Its Derivative. Diakses dari: https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/

LibreTexts. Calculus: The Chain Rule for Multivariable Functions. Diakses dari: https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions

NumPy Developers. NumPy 2.2 Documentation. Diakses dari: https://numpy.org/doc/2.2/

Orr, D. (2021). A Light Introduction to Automatic Differentiation. Diakses dari: https://douglasorr.github.io/2021-11-autodiff/article.html

Osajima, J. The spelled-out intro to neural networks and backpropagation: building micrograd (Forward Propagation). Diakses dari: https://www.jasonosajima.com/forwardprop

Osajima, J. The spelled-out intro to neural networks and backpropagation: building micrograd (Backpropagation). Diakses dari: https://www.jasonosajima.com/backprop

scikit-learn developers. sklearn.neural_network.MLPClassifier Documentation. Diakses dari: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Raka Daffa Iftikhaar (13523018)
Muhammad Fithra Rizki (13523049)
Muhammad Timur Kanigara (13523055)

Fakultas/Sekolah : STEI
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 1 Pembelajaran Mesin

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	Tingkat 1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	Tingkat 1
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Tingkat 2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	Tingkat 1
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Maret 2026

Raka Daffa Iftikhaar	Muhammad Fithra Rizki	Muhammad Timur Kanigara
13523018	13523049	13523055